BANKING MANAGEMENT SYSTEM

Software Requirements Specification

Scope, actors, functional and non-functional requirements

CS-513 • System Software Mini Project

Source baseline: repository main branch + supplied source archive

Prepared 25 October 2026 | Version 1.0

https://github.com/Dev2907/Banking-Management-IIITB

SRS

1. Purpose

This SRS records the intended scope of the CS-513 Banking Management System and distinguishes the assignment requirements from features present in the current code. It is intended to guide implementation review, testing, and project demonstration.

2. Product Scope

The system provides terminal-based banking operations for customers and administrative workflows for bank employees, managers, and administrators. A TCP client communicates with a local server, which owns the SQLite database and enforces role-sensitive operations.

3. User Classes

Actor	Authority / needs
Customer (role 1)	Own accounts, balance and history; make deposits, withdrawals and transfers; submit loan requests and feedback; change password.
Employee (role 2)	Maintain selected customer identity/active state; work on assigned loans; inspect users/accounts and customer transactions.
Manager (role 3)	Maintain selected customer identity/active state; assign pending applications to active employees; inspect loans, users/accounts and feedback.
Administrator (role 4)	Create non-administrator user records, modify a user role/username/active state, and change password.

4. Operating Environment and Constraints

- Linux/POSIX environment with a C++17-capable g++ compiler.
- SQLite development library and sqlite3 CLI are required for compiling and database initialization.
- libsodium development headers/library are required for password hashing.
- Run server and client from the project root because DB and lock paths are relative.
- Server binds only to IPv4 loopback 127.0.0.1:8080 in current source.
- Money amounts are integers in smallest currency units; the command-line parsing does not accept decimal-point money strings.
- The schema initialization script drops and recreates tables; existing data must be backed up first.

5. Functional Requirements

ID	Requirement	Description	Status
FR-01	Authentication	The system shall authenticate a username/password pair and verify the requested role against the stored role.	Implemented
FR-02	Active accounts	Inactive users shall be rejected during login.	Implemented
FR-03	Single session	The system shall reject a second simultaneous login for the same user ID using a per-user file lock.	Implemented
FR-04	Customer signup	A customer may sign up; user creation and initial zero-balance account creation shall be atomic.	Implemented
FR-05	Account selection	A customer may view owned accounts and select an account for the session.	Implemented

ID	Requirement	Description	Status
FR-06	Balance and new account	A customer may view the selected account balance and create an additional account.	Implemented
FR-07	Deposit	A customer may deposit a positive amount; balance and transaction history shall update atomically.	Implemented
FR-08	Withdrawal	A customer may withdraw a positive amount only when funds are sufficient; balance and transaction history shall update atomically.	Implemented
FR-09	Transfer	A customer may transfer funds to an active customer by username; source debit, recipient credit, and paired transaction records shall be atomic.	Implemented
FR-10	Transaction history	A customer may inspect recent transaction records for the selected account.	Implemented; current query returns up to 10
FR-11	Loan request	A customer may submit a positive loan amount for manager assignment and later employee review.	Implemented
FR-12	Feedback	A customer may submit feedback; a manager may review recent feedback.	Implemented
FR-13	Password change	An authenticated user may change password after verifying the current password; the new password shall be hashed.	Implemented for all four roles
FR-14	Employee customer maintenance	An employee shall create and modify customer records as specified by the assignment.	username and user active state can be changed;
FR-15	Employee loan processing	An employee shall view loans assigned to them and approve or reject a loan assigned to them.	Implemented; approval credits the loan amount to the account
FR-16	Employee history / overview	An employee shall view customer transaction history and users/accounts.	Implemented
FR-17	Manager customer controls	A manager shall activate/deactivate customer accounts.	semantic match: current operation changes users.is_active (login status)
FR-18	Manager loan assignment	A manager shall assign pending loan applications to active employees.	Implemented
FR-19	Administrator user creation	An administrator shall add an employee and manage users/roles.	Implemented through createUser(role=2) and modifyUser(); role creation is limited to 1..3
FR-20	Logout	Each role shall be able to terminate its authenticated session.	Implemented; session-lock release on all exits should be tested

6. Non-Functional Requirements

ID	Requirement
NFR-01 Security	Store password hashes via libsodium; use bound SQL parameters for user-controlled values; enforce role checks in server-side code.
NFR-02 Integrity	Reject invalid positive/negative amounts and prevent overdrafts; use DB transactions for multi-step financial changes.
NFR-03 Concurrency	Allow multiple client connections via separate child processes and restrict each user to one active session via flock.
NFR-04 Durability	Committed state is stored in SQLite; confirm persistence with a restart acceptance test.
NFR-05 Usability	All functionality is available through terminal prompts and textual responses.

ID	Requirement
NFR-06 Portability	The implementation targets Linux/POSIX and requires C++17 plus SQLite and libsodium development packages.
NFR-07 Maintainability	Role business logic is separated into authService and four portal classes, with shared password/database helpers.
NFR-08 Network scope	The current server only listens on loopback and the TCP stream is not encrypted.
NFR-09 Observability	Server writes some operational/error messages to stdout/stderr; structured audit logging is not implemented.

7. Role / Permission Matrix

Operation	Customer	Employee	Manager	Admin
Account balance and self transactions	Yes	No	No	No
Deposit / withdraw / transfer	Yes	No	No	No
Loan apply	Yes	No	No	No
View assigned loans / approve or reject	No	Yes	No	No
Assign pending loan to employee	No	No	Yes	No
View loans / users and accounts	No	Yes*	Yes	No
Review customer feedback	No	No	Yes	No
Change customer username / active state	No	Yes	Yes	Via modify user
Create user	Customer self-signup	No	No	Roles 1–3
Modify username / role / active state	No	No	No	Yes
Change own password / logout	Yes	Yes	Yes	Yes

*The employee user/account overview excludes manager-role users in the current query. The manager overview excludes manager-role users as well.

8. Requirements Traceability to Assignment Brief

Assignment item	Evidence in current baseline	Status
Blueprint/class diagram	Module/class diagram and ERD included in this documentation pack.	Provided
Four role modules	Customer, employee, manager and administrator portal modules exist.	Implemented
Socket programming / multiple clients	TCP sockets, accept loop and fork-per-client.	Implemented; concurrency test evidence outstanding
Process/file management and file locking	fork(), open(), close(), flock(), and waitpid() used.	Implemented; child-reaping robustness needs improvement
Race-condition handling	Session flock and SQLite write transactions.	Implement

Assignment item	Evidence in current baseline	Status
ACID operations	Deposit, withdraw, transfer, signup/account creation, loan apply/decision use transactions in implementation paths.	Implemented in code; evidence tests outstanding
Employee add customer	Create-customer is available through customer signup/admin path, not employee portal.	Gap against brief wording

9. Acceptance Conditions

- All four roles can sign in, perform authorized operations, change password, and log out.
- Unauthorized role/ownership attempts fail at the server rather than only being hidden in the client menu.
- Transfers preserve both account balances and both history records on commit, or preserve none on failure.
- Insufficient-fund withdrawals and transfers do not reduce balances.
- A second simultaneous session for a user is rejected; the lock is available after logout or process termination.
- Committed balances and history persist after a server restart.
- The implementation gaps listed in this SRS are resolved or explicitly accepted by the instructor/project rubric.